Generated Document

گزارش نهایی پروژه دوم آزمایشگاه سیستم‌عامل

معراج رستگار 810101576 - علی صادقی 810102471 - معراج پورحسینی 810102420

فهرست مطالب

1 هدف و خلاصه پروژه

2 فایل‌های تغییر داده‌شده و اضافه‌شده

2-1 فایل‌های تغییر داده‌شده

2-2 فایل‌های اضافه‌شده

3 سازوکار فراخوانی سیستمی در xv6

4 پاسخ پرسش‌های نظری مقدماتی

4-1 پرسش ۱: کتابخانه‌های سطح کاربر چگونه جزئیات فراخوانی سیستمی را پنهان می‌کنند؟

4-2 پرسش ۲: غیر از system call چه روش‌هایی برای تعامل برنامه‌های کاربر با kernel در Linux وجود دارد؟

4-3 پرسش ۳: چرا فقط system call با سطح دسترسی USER_DPL فعال می‌شود؟

4-4 پرسش ۴: چرا ss و esp فقط هنگام تغییر سطح دسترسی روی پشته push می‌شوند؟

4-5 پرسش ۵: نقش argint و argptr چیست و چرا بررسی بازه آدرس ضروری است؟

5 بررسی گام‌های اجرای system call با GDB

5-1 برنامه تست getpidtest.c

5-2 روش اجرای GDB

5-3 دستور bt و تحلیل خروجی

5-4 دستور down و مشاهده eax در trapframe

6 بخش اول عملی: مولد اعداد شبه‌تصادفی

6-1 خواسته پروژه

6-2 پیاده‌سازی انجام‌شده در sysproc.c

6-3 برنامه تست تاس مجازی dice.c

6-4 پاسخ پرسش‌های بخش random

6-4-1 پرسش ۱: مولد اعداد تصادفی در Linux و Windows چگونه است و چرا در kernel پیاده‌سازی می‌شود؟

6-4-2 پرسش ۲: اگر دو بار پشت سر هم setSeed اجرا شود، خروجی تغییر می‌کند؟ چرا؟

7 بخش دوم عملی: نمایش اطلاعات پردازه

7-1 خواسته پروژه

7-2 پیاده‌سازی انجام‌شده

7-3 برنامه تست pinfo_test.c

7-4 پاسخ پرسش‌های بخش اطلاعات پردازه

7-4-1 پرسش ۱: stateهای پردازه در xv6 چیست؟

7-4-2 پرسش ۲: اگر پردازه‌ای شناسه خودش را بدهد، در قسمت state چه نتیجه‌ای نمایش داده می‌شود؟

7-4-3 پرسش ۳: آیا می‌توانستیم process_information را در فضای کاربر پیاده‌سازی کنیم؟ مزایا و معایب چیست؟

8 بخش سوم عملی: مرتب‌سازی در هسته و فضای کاربر

8-1 خواسته پروژه

8-2 پیاده‌سازی sys_sort_numbers در sysfile.c

8-3 برنامه تست kernel sort: sort_kernel.c

8-4 پیاده‌سازی همان الگوریتم در user space: sort_user.c

8-5 قالب فایل ورودی و خروجی

8-6 پاسخ پرسش‌های بخش sort

8-6-1 پرسش ۱: تعداد تغییر mode بین فضای کاربر و هسته در هر بخش چقدر است؟

8-6-1-1 حالت kernel sort

8-6-1-2 حالت user-space sort

8-6-2 پرسش ۲: آیا لازم بود sort در فضای هسته پیاده‌سازی شود؟

8-6-3 پرسش ۳: دستور sort در Linux در فضای کاربر پیاده‌سازی شده یا فضای هسته؟

8-6-4 پرسش ۴: kernel مسئول چه کارهایی است؟

8-6-5 پرسش ۵: چرا نباید از الگوریتم‌های مرتب‌سازی بازگشتی استفاده کنیم؟

9 دستورهای ساخت و اجرای پروژه

9-1 ساخت پروژه

9-2 اجرای xv6

9-3 تست‌های پیشنهادی داخل xv6

10 جمع‌بندی

11 منابع

هدف و خلاصه پروژه

در این پروژه مسیر کامل ورود یک برنامه سطح کاربر به هسته xv6 از طریق فراخوانی سیستمی بررسی شد. در xv6، برنامه کاربر مستقیماً وارد هسته نمی‌شود؛ بلکه از wrapperهای سطح کاربر استفاده می‌کند. wrapper شماره system call را در ثبات `eax` قرار می‌دهد، دستور `int $T_SYSCALL` را اجرا می‌کند، و پردازنده با کمک جدول IDT و Trap Gate مربوط به شماره ۶۴ کنترل را به هسته می‌دهد. در هسته، کد اسمبلی `alltraps` ثبات‌ها را ذخیره می‌کند، تابع `trap()` نوع trap را تشخیص می‌دهد، و در نهایت `syscall()` تابع مناسب را از جدول `syscalls` صدا می‌زند.

در پروژه نهایی چهار فراخوانی سیستمی جدید اضافه شده است:

```c
int setSeed(void);
int getRandomNumber(int n, int *buf);
int process_information(int pid);
int sort_numbers(const char *src_file);
```

برای تست این فراخوانی‌ها نیز پنج برنامه سطح کاربر نوشته شده است:

هدف	برنامه
اجرای ساده `getpid()` برای بررسی مسیر system call با GDB	`getpidtest.c`
شبیه‌سازی تاس مجازی با `setSeed` و `getRandomNumber`	`dice.c`
ساخت چند فرزند و تست اطلاعات پردازه، فرزندان و siblingها	`pinfo_test.c`
اجرای `sort_numbers` در هسته و اندازه‌گیری زمان	`sort_kernel.c`
اجرای همان الگوریتم مرتب‌سازی در فضای کاربر و اندازه‌گیری زمان	`sort_user.c`

فایل‌های تغییر داده‌شده و اضافه‌شده

فایل‌های تغییر داده‌شده

تغییر انجام‌شده	فایل
اضافه شدن برنامه‌های `_dice`، `_getpidtest`، `_pinfo_test`، `_sort_kernel` و `_sort_user` به `UPROGS` و `EXTRA`؛ اضافه شدن گزینه‌های سازگاری کامپایل مانند `-Wno-array-bounds`	`Makefile`
تعریف شماره‌های جدید `SYS_setSeed = 22`، `SYS_getRandomNumber = 23`، `SYS_process_information = 24` و `SYS_sort_numbers = 25`	`syscall.h`
معرفی `extern` برای system callهای جدید و اضافه شدن آن‌ها به آرایه `syscalls`	`syscall.c`
اضافه شدن prototypeهای سطح کاربر برای system callهای جدید	`user.h`
اضافه شدن wrapperهای `SYSCALL(setSeed)`، `SYSCALL(getRandomNumber)`، `SYSCALL(process_information)` و `SYSCALL(sort_numbers)`	`usys.S`
اضافه شدن prototype تابع kernel-level `procinfo`	`defs.h`
پیاده‌سازی `sys_setSeed`، `sys_getRandomNumber` و `sys_process_information`	`sysproc.c`
پیاده‌سازی `procinfo` و جدول نام وضعیت‌های پردازه برای چاپ state	`proc.c`
پیاده‌سازی `sys_sort_numbers` و تابع‌های کمکی مربوط به خواندن، sort، parse و نوشتن خروجی	`sysfile.c`

فایل‌های اضافه‌شده

توضیح	فایل
برنامه ساده برای فراخوانی `getpid()` و استفاده در GDB	`getpidtest.c`
دریافت تعداد تاس از آرگومان خط فرمان و چاپ اعداد ۱ تا ۶	`dice.c`
ایجاد سه فرزند، چاپ اطلاعات فرزندان و والد	`pinfo_test.c`
اجرای `sort_numbers(argv[1])` و چاپ زمان بر حسب tick	`sort_kernel.c`
خواندن فایل، parse اعداد، insertion sort در user space، و نوشتن خروجی	`sort_user.c`

سازوکار فراخوانی سیستمی در xv6

مسیر اجرای system call در xv6 را می‌توان به این صورت خلاصه کرد:

1. برنامه کاربر، تابعی مثل `getpid()` یا `sort_numbers()` را صدا می‌زند.
2. برنامه wrapper ساخته‌شده در `usys.S` اجرا می‌شود.
3. برنامه wrapper شماره system call را در `%eax` قرار می‌دهد.
4. دستور `int $T_SYSCALL` اجرا می‌شود.
5. پردازنده با استفاده از IDT و Trap Gate شماره ۶۴ وارد kernel mode می‌شود.
6. بردار `vector64` در `vectors.S` ابتدا error code صفر و trap number را push می‌کند و سپس به `alltraps` می‌پرد.
7. `alltraps` ثبات‌ها را push می‌کند و trapframe را می‌سازد.
8. تابع `trap(tf)` صدا زده می‌شود.
9. اگر `tf->trapno == T_SYSCALL` باشد، تابع `syscall()` اجرا می‌شود.
10. `syscall()` مقدار `tf->eax` را به عنوان شماره system call می‌خواند.
11. تابع مناسب از جدول `syscalls` اجرا می‌شود.
12. مقدار بازگشتی در `tf->eax` قرار می‌گیرد و با `iret` کنترل به user mode برمی‌گردد.

نمونه wrapper در `usys.S`:

```asm
#define SYSCALL(name) \
  .globl name; \
  name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret
```

این قطعه یکی از مهم‌ترین بخش‌های پنهان‌سازی جزئیات system call است: برنامه‌نویس سطح کاربر فقط یک تابع C صدا می‌زند، اما در پشت صحنه شماره system call، trap و بازگشت از هسته انجام می‌شود.

پاسخ پرسش‌های نظری مقدماتی

پرسش ۱: کتابخانه‌های سطح کاربر چگونه جزئیات فراخوانی سیستمی را پنهان می‌کنند؟

در xv6، فایل `usys.S` با ماکروی `SYSCALL(name)` برای هر system call یک wrapper کوچک اسمبلی تولید می‌کند. این wrapper شماره system call را از `syscall.h` برمی‌دارد، آن را در ثبات `eax` قرار می‌دهد، سپس با `int $T_SYSCALL` وارد kernel می‌شود. بنابراین برنامه‌نویس نیازی ندارد شماره system call، شماره trap، قرارداد registerها یا جزئیات بازگشت از trap را بداند.

فایل‌های کتابخانه‌ای سطح کاربر مانند `ulib.c` و `printf.c` هم یک لایه انتزاعی بالاتر می‌سازند. مثلاً برنامه‌نویس `printf` را صدا می‌زند، اما در نهایت این تابع از `write` استفاده می‌کند. بنابراین جزئیات trap، file descriptor و انتقال کنترل به kernel از دید برنامه‌نویس پنهان می‌شود.

مزیت‌های این انتزاع:

- **قابلیت حمل:** اگر مکانیزم ورود به kernel در یک معماری تغییر کند، فقط wrapperها تغییر می‌کنند و برنامه‌های کاربر بدون تغییر باقی می‌مانند.
- **امنیت:** همه ورودها از نقاط کنترل‌شده و مشخص انجام می‌شود و kernel می‌تواند آرگومان‌ها را بررسی کند.
- **سادگی توسعه:** برنامه‌نویس با امضای آشنای تابع‌های C کار می‌کند، نه با trap number و ثبات‌ها.
- **پایداری ABI:** قرارداد بین user space و kernel متمرکز و قابل مدیریت می‌ماند.

پرسش ۲: غیر از system call چه روش‌هایی برای تعامل برنامه‌های کاربر با kernel در Linux وجود دارد؟

هرچند بسیاری از این روش‌ها در نهایت از system callهای پایه مثل `read`، `write`، `ioctl` یا `mmap` استفاده می‌کنند، از دید برنامه‌نویس رابط‌های مستقلی برای تعامل با kernel هستند:

1. **فایل‌های دستگاهی در `/dev`:** فایل‌هایی مثل `/dev/random`، `/dev/urandom`، دیسک‌ها و ترمینال‌ها نماینده device driverهای kernel هستند. برنامه با عملیات فایل معمولی با آن‌ها کار می‌کند.
2. **`procfs` و `sysfs`:** مسیرهایی مثل `/proc` و `/sys` اطلاعات kernel، پردازه‌ها و deviceها را به شکل فایل‌های مجازی ارائه می‌کنند.
3. **`ioctl`:** رابط عمومی برای ارسال فرمان‌های خاص به driverها. بسیاری از تنظیمات deviceها با `ioctl` انجام می‌شود.
4. **بخش Netlink socket:** مکانیزمی برای ارتباط ساختاریافته user space و kernel، مخصوصاً در پیکربندی شبکه.
5. **`mmap`:** نگاشت فایل، device یا حافظه مشترک در فضای آدرس user. پس از map شدن، بخشی از تعامل بدون trapهای مکرر انجام می‌شود.

6. **بخش Signalها:** kernel می‌تواند رخدادها را به صورت asynchronous به پردازه اطلاع دهد؛ مانند `SIGCHLD`، `SIGSEGV` یا `SIGINT`.
7. **بخش vDSO:** برای برخی عملیات پرتکرار مثل دریافت زمان، kernel قطعه کدی را در فضای آدرس کاربر map می‌کند تا در موارد ممکن بدون ورود مستقیم به kernel اجرا شود.

پرسش ۳: چرا فقط system call با سطح دسترسی `USER_DPL` فعال می‌شود؟

در x86، ورودی‌های IDT از نوع Gate Descriptor هستند و فیلد DPL مشخص می‌کند کدی با چه سطح دسترسی اجازه دارد آن gate را با دستور `int` فعال کند. system call باید از user mode قابل اجرا باشد؛ بنابراین Trap Gate مربوط به `T_SYSCALL` با `DPL_USER` تنظیم می‌شود.

اما وقفه‌های سخت‌افزاری و exceptionها نباید مستقیماً توسط برنامه کاربر ساخته شوند. اگر DPL آن‌ها هم user-level بود، یک برنامه مخرب می‌توانست با `int n` وقفه سخت‌افزاری جعلی یا exception مصنوعی ایجاد کند و handlerهایی را فعال کند که برای رخداد واقعی طراحی شده‌اند. این کار می‌تواند باعث خراب شدن وضعیت kernel، دور زدن مسیرهای امنیتی یا denial of service شود. بنابراین فقط مسیر system call به‌صورت کنترل‌شده برای user باز است و بقیه trapها در سطح kernel باقی می‌مانند.

پرسش ۴: چرا `ss` و `esp` فقط هنگام تغییر سطح دسترسی روی پشته push می‌شوند؟

وقتی trap از user mode به kernel mode رخ می‌دهد، CPU باید stack را عوض کند؛ یعنی از user stack به kernel stack مشخص‌شده در TSS برود. برای اینکه بعداً با `iret` بتواند دقیقاً به user stack قبلی برگردد، پردازنده مقدارهای `ss` و `esp` کاربر را روی kernel stack ذخیره می‌کند.

اما اگر trap در همان سطح دسترسی رخ دهد، مثلاً kernel خودش دچار trap شود، stack تغییر نمی‌کند. چون stack همان stack فعلی است، نیازی به ذخیره `ss` و `esp` نیست. به همین دلیل سخت‌افزار فقط هنگام privilege change این دو مقدار را push می‌کند.

پرسش ۵: نقش `argint` و `argptr` چیست و چرا بررسی بازه آدرس ضروری است؟

در xv6 آرگومان‌های system call طبق قرارداد C روی stack برنامه کاربر قرار می‌گیرند. هنگام ورود به kernel، مقدار `esp` کاربر در trapframe ذخیره شده است. تابع‌هایی مثل `argint`، `argstr` و `argptr` با استفاده از همین `tf->esp` آرگومان‌ها را از فضای کاربر بازیابی می‌کنند.

- `argint(n, &x)` آرگومان nام را به صورت عدد صحیح می‌خواند.
- `argstr(n, &s)` آرگومان nام را به عنوان رشته کاربر بررسی می‌کند.
- `argptr(n, &p, size)` ابتدا آدرس ارسالی از user را می‌خواند و سپس بررسی می‌کند که کل بازه `[p, p + size)` داخل فضای آدرس معتبر همان پردازه باشد.

اهمیت `argptr` امنیتی است. اگر kernel بدون بررسی، pointer کاربر را قبول کند، کاربر می‌تواند آدرس نامعتبر یا حتی آدرسی نزدیک به فضای kernel بدهد. برای مثال، در `sys_read(fd, buf, n)` اگر `buf` بررسی نشود، برنامه کاربر می‌تواند کاری کند که داده خوانده‌شده روی حافظه kernel نوشته شود یا باعث page fault داخل kernel شود. نتیجه ممکن است crash سیستم، خراب شدن ساختارهای kernel، نشت اطلاعات یا آسیب‌پذیری امنیتی باشد. پس `argptr` مرز حفاظتی بین user space و kernel space را enforce می‌کند.

بررسی گام‌های اجرای system call با GDB

برنامه تست `getpidtest.c`

برای بررسی مسیر system call، برنامه زیر به پروژه اضافه شده است:

```c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
  int pid = getpid();
  printf(1, "getpid() -> %d\n", pid);
  exit();
}
```

روش اجرای GDB

در ترمینال اول:

```bash
make CPUS=1 qemu-nox-gdb
```

در ترمینال دوم:

```bash
gdb kernel
(gdb) target remote :26000
(gdb) b syscall
(gdb) c
```

سپس داخل xv6:

```sh
getpidtest
```

دستور `bt` و تحلیل خروجی

بعد از توقف روی breakpoint تابع `syscall()`، دستور زیر اجرا می‌شود:

```gdb
(gdb) bt
```

نمونه خروجی مورد انتظار:

```gdb
#0  syscall () at syscall.c:...
#1  trap (tf=0x...) at trap.c:...
#2  alltraps () at trapasm.S:...
#3  0x00000... in ?? ()
```

تحلیل:

- در frame شماره `#0` نشان می‌دهد اکنون داخل تابع `syscall()` هستیم.
- در frame شماره `#1` تابع `trap()` است؛ یعنی trap رخ داده، `trap()` آن را بررسی کرده و چون `trapno` برابر `T_SYSCALL` بوده، `syscall()` را صدا زده است.
- در frame شماره `#2` مربوط به `alltraps` است؛ این بخش اسمبلی ثبات‌ها را ذخیره کرده و trapframe را ساخته است.
- در frame بعدی آدرس برگشت در فضای کاربر است؛ همان جایی که wrapper سطح کاربر دستور `int $T_SYSCALL` را اجرا کرده است.

دستور `bt` مخفف backtrace است و زنجیره فراخوانی‌ها یا frameهای پشته را از نقطه فعلی تا callerهای قبلی نشان می‌دهد. این دستور برای دیدن مسیر دقیق ورود از user به kernel بسیار مفید است.

دستور `down` و مشاهده `eax` در trapframe

دستورهای `up` و `down` در GDB برای جابه‌جایی بین frameهای backtrace استفاده می‌شوند. `up` معمولاً به سمت callerها و frameهای بیرونی‌تر می‌رود و `down` در جهت عکس، به سمت frameهای داخلی‌تر برمی‌گردد. چون متغیر `tf` پارامتر تابع `trap()` است، برای چاپ آن باید frame مربوط به `trap` انتخاب شود؛ مثلاً:

```
GDB

(gdb) frame 1
(gdb) p tf->eax
```

یا اگر ابتدا با `up` به frame مربوط به `trap` رفته باشیم، می‌توانیم با `down` به frame داخلی‌تر برگردیم. نکته اصلی این است که مقدار شماره system call در `tf->eax` ذخیره شده است.

برای `getpid` مقدار مورد انتظار برابر شماره تعریف‌شده در `syscall.h` است:

```
C

#define SYS_getpid 11
```

پس خروجی مورد انتظار:

```
GDB

$1 = 11
```

اگر در اولین توقف، مقدار دیگری دیده شود، علت این است که قبل از اجرای `getpidtest` یا قبل از رسیدن به system call موردنظر، shell یا برنامه‌های دیگر system callهایی مثل `read`، `write`، `fork` یا `exec` اجرا کرده‌اند. با چند بار اجرای دستور `c`، در نهایت به فراخوانی `getpid` می‌رسیم و مقدار `tf->eax` برابر `11` مشاهده می‌شود.

بخش اول عملی: مولد اعداد شبه‌تصادفی

خواسته پروژه

دو system call زیر باید به xv6 اضافه شود:

```c
int setSeed(void);
int getRandomNumber(int n, int *buf);
```

`setSeed` باید با استفاده از وضعیت فعلی سیستم seed را بسازد:

```text
seed = (ticks << 20) ^ (pid << 10) ^ (stack_addr & 0x3FF)
```

`getRandomNumber` باید با استفاده از state فعلی، `n` عدد تولید کند و در بافر کاربر قرار دهد. شرط پروژه این است که `n < 15` باشد. فرمول تولید عدد بعدی:

```text
random_state = (random_state * 1103515245 + 12345) & 0x7fffffff
```

در صورت موفقیت هر دو system call مقدار `0` و در صورت خطا مقدار `-1` برمی‌گردانند.

پیاده‌سازی انجام‌شده در `sysproc.c`

برای نگهداری state مولد، چند متغیر static در kernel تعریف شده است:

```c
static struct spinlock randlock;
static int randinit;
static uint random_state;
```

تابع `randominit_once` تضمین می‌کند lock فقط یک‌بار مقداردهی شود:

```c
static void
randominit_once(void)
{
  acquire(&tickslock);
  if(randinit == 0){
    initlock(&randlock, "randlock");
    randinit = 1;
  }
  release(&tickslock);
}
```

در `sys_setSeed` مقدار `ticks` با `tickslock` خوانده می‌شود، pid از `myproc()->pid` گرفته می‌شود، آدرس یک متغیر محلی kernel به عنوان `stack_addr` استفاده می‌شود، سپس `random_state` با `randlock` به‌روزرسانی می‌شود.

در `sys_getRandomNumber` ابتدا آرگومان‌ها با `argint` و `argptr` گرفته و بررسی می‌شوند. اگر `n <= 0` یا `n >= 15` باشد، مقدار `-1` برگردانده می‌شود. سپس داخل ناحیه محافظت‌شده با `randlock`، state طبق فرمول پروژه به‌روزرسانی می‌شود. برای نوشتن امن در بافر کاربر، از `copyout` استفاده شده است:

```c
if(copyout(myproc()->pgdir, (uint)&ubuf[i], &value, sizeof(value)) < 0){
  release(&randlock);
  return -1;
}
```

استفاده از `copyout` باعث می‌شود نوشتن در فضای کاربر با page table همان پردازه و با کنترل خطا انجام شود.

برنامه تست تاس مجازی `dice.c`

برنامه `dice` تعداد تاس را از آرگومان خط فرمان می‌گیرد. چون در پروژه فرض شده `n < 15`، برنامه تعداد را در بازه `1..14` قبول می‌کند. سپس ابتدا `setSeed()` و بعد `getRandomNumber(rolls, values)` را صدا می‌زند و هر عدد تصادفی را به بازه ۱ تا ۶ نگاشت می‌کند:

```c
r = (values[i] % 6) + 1;
```

روش اجرا:

```sh
dice 10
```

نمونه خروجی:

```text
roll 1: 4
roll 2: 1
roll 3: 6
...
```

پاسخ پرسش‌های بخش random

پرسش ۱: مولد اعداد تصادفی در Linux و Windows چگونه است و چرا در kernel پیاده‌سازی می‌شود؟

در Linux، رابط‌هایی مثل `/dev/urandom` و system call جدیدتر `getrandom()` خروجی CSPRNG هسته را در اختیار برنامه‌ها می‌گذارند. مستندات Linux می‌گوید `getrandom()` بافر کاربر را با بایت‌های تصادفی مناسب برای seed کردن PRNGهای user-space یا کاربردهای رمزنگاری پر می‌کند و به طور پیش‌فرض از منبع urandom استفاده می‌کند. همچنین `random(7)` توضیح می‌دهد که kernel RNG از entropy جمع‌آوری‌شده از driverها و منابع محیطی برای seed کردن CSPRNG استفاده می‌کند.

در Windows، تابع رسمی `BCryptGenRandom` برای دریافت random امن استفاده می‌شود. حالت `BCRYPT_USE_SYSTEM_PREFERRED_RNG` از مولد ترجیحی سیستم استفاده می‌کند و برنامه نیازی به مدیریت مستقیم الگوریتم و state ندارد.

دلیل پیاده‌سازی یا مدیریت RNG امن در سطح سیستم‌عامل:

- بخش kernel به رخدادهای سخت‌افزاری، وقفه‌ها، timing و deviceها دسترسی دارد و بهتر می‌تواند entropy جمع کند.
- باید state داخلی RNG از دستکاری برنامه‌های کاربر محافظت شود.
- همه برنامه‌ها به یک منبع مرکزی و قابل اعتماد دسترسی دارند.
- برای کاربردهای حساس مثل تولید کلید، randomness ضعیف در user space می‌تواند خطر امنیتی جدی ایجاد کند.

پرسش ۲: اگر دو بار پشت سر هم `setSeed` اجرا شود، خروجی تغییر می‌کند؟ چرا؟

فرمول seed به سه مؤلفه وابسته است: `ticks`، `pid` و بخش پایینی آدرس یک متغیر محلی روی kernel stack. اگر دو بار پشت سر هم در همان پردازه و در فاصله‌ای بسیار کوتاه `setSeed` اجرا شود، معمولاً `pid` ثابت است، آدرس

متغیر محلی نیز غالباً مشابه است، و ممکن است `ticks` هنوز تغییر نکرده باشد. در این حالت seed یکسان می‌شود و دنباله خروجی بعدی نیز یکسان خواهد بود.

اما اگر بین دو فراخوانی، وقفه timer رخ دهد و `ticks` تغییر کند، seed متفاوت می‌شود و دنباله خروجی نیز تغییر می‌کند. بنابراین تغییر خروجی تضمینی نیست، بلکه به تغییر وضعیت سیستم، مخصوصاً `ticks`، بستگی دارد.

بخش دوم عملی: نمایش اطلاعات پردازه

خواسته پروژه

باید system call زیر پیاده‌سازی شود:

```c
int process_information(int pid);
```

این فراخوانی باید اطلاعات پردازه‌ای با pid داده‌شده را مستقیماً از جدول پردازه‌های kernel استخراج و با قالب زیر چاپ کند:

```text
Process_id:
Parent_id:
Children's_id: child1, child2, ..., child n
Siblings_id: sibling1, sibling2, ..., sibling n
Depth:
State:
```

اگر pid پیدا نشود، مقدار `-1` و در صورت موفقیت مقدار `0` برگردانده می‌شود.

پیاده‌سازی انجام‌شده

در `sysproc.c` تابع wrapper هسته‌ای `sys_process_information` نوشته شده است:

```c
int
sys_process_information(void)
{
  int pid;

  if(argint(0, &pid) < 0)
    return -1;
  return procinfo(pid);
}
```

تابع اصلی در `proc.c` با نام `procinfo` پیاده‌سازی شده است. این تابع ابتدا `ptable.lock` را می‌گیرد تا جدول پردازه‌ها هنگام پیمایش تغییر نکند. سپس پردازه هدف را پیدا می‌کند و این اطلاعات را چاپ می‌کند:

- `Process_id` : مقدار `p->pid`
- `Parent_id` : اگر parent وجود داشته باشد، `p->parent->pid` وگرنه `-1`
- `Children's_id` : همه پردازه‌هایی که `q->parent == p` و state آن‌ها `UNUSED` نیست
- `Siblings_id` : همه پردازه‌هایی که parent مشترک با پردازه هدف دارند و خود پردازه هدف نیستند
- `Depth` : تعداد parentهایی که از پردازه هدف تا ریشه طی می‌شود
- `State` : نام state با استفاده از آرایه `pstate_names`

برای stateها، آرایه‌ای مانند زیر در `proc.c` اضافه شده است:

```c
static char *pstate_names[] = {
[UNUSED]    "unused",
[EMBRYO]    "embryo",
[SLEEPING]  "sleep ",
[RUNNABLE]  "runble",
[RUNNING]   "run   ",
[ZOMBIE]    "zombie"
};
```

برنامه تست `pinfo_test.c`

برنامه تست با تابع `spawn_level(3)` سه فرزند ایجاد می‌کند. هر فرزند پس از `sleep(200)` اطلاعات خودش را با `process_information(getpid())` چاپ می‌کند. والد نیز بعد از `sleep(30)` اطلاعات خود را چاپ کرده و سپس با سه بار `wait()` منتظر پایان فرزندان می‌ماند.

روش اجرا:

```sh
pinfo_test
```

نمونه خروجی مورد انتظار:

```text
=== parent info ===
Process_id: 4
Parent_id: 2
Children's_id: 5, 6, 7
Siblings_id: none
Depth: 2
State: run

Process_id: 5
Parent_id: 4
Children's_id: none
Siblings_id: 6, 7
Depth: 3
State: run
```

مقادیر pid در هر اجرا ممکن است متفاوت باشد.

پاسخ پرسش‌های بخش اطلاعات پردازه

پرسش ۱: stateهای پردازه در xv6 چیست؟

state	توضیح
`UNUSED`	خانه‌ای از جدول پردازه‌ها آزاد است و پردازه‌ای در آن قرار ندارد.
`EMBRYO`	پردازه در حال ساخته شدن است و هنوز آماده اجرا نیست.
`SLEEPING`	پردازه منتظر رخدادی مثل I/O، تایمر یا wakeup است.
`RUNNABLE`	پردازه آماده اجراست ولی فعلاً CPU به آن اختصاص داده نشده است.
`RUNNING`	پردازه هم‌اکنون روی CPU در حال اجراست.
`ZOMBIE`	پردازه `exit` کرده ولی parent هنوز `wait` نکرده است.

پرسش ۲: اگر پردازه‌ای شناسه خودش را بدهد، در قسمت state چه نتیجه‌ای نمایش داده می‌شود؟

اگر پردازه خودش `process_information(getpid())` را صدا بزند، همان پردازه در حال اجرای system call است و CPU در اختیار آن قرار دارد. بنابراین state آن معمولاً `RUNNING` نمایش داده می‌شود. در پروژه نهایی به دلیل آرایه نام‌ها، این حالت با رشته `run` یا مشابه آن چاپ می‌شود.

پرسش ۳: آیا می‌توانستیم `process_information` را در فضای کاربر پیاده‌سازی کنیم؟ مزایا و معایب چیست؟

در xv6 معمولی، پیاده‌سازی کامل این قابلیت در user space ممکن نیست، چون جدول پردازه‌ها (`ptable`)، parent pointerها و state دقیق پردازه‌ها فقط در kernel وجود دارند و user space دسترسی مستقیم به آن‌ها ندارد. برنامه کاربر فقط می‌تواند pid خودش یا pid فرزندانی را که ساخته تا حدی نگه دارد، اما نمی‌تواند state واقعی و کامل همه پردازه‌ها را دقیق بخواند.

مقایسه:

روش	مزایا	معایب
پیاده‌سازی در kernel	دسترسی مستقیم به `ptable`، امکان قفل کردن جدول، اطلاعات دقیق و کامل	افزایش کد kernel؛ bug در kernel می‌تواند کل سیستم را خراب کند
پیاده‌سازی در user space	امن‌تر و قابل تغییرتر؛ خرابی برنامه فقط همان برنامه را تحت تأثیر می‌گذارد	در xv6 بدون رابطی مثل `/proc` اطلاعات کامل در دسترس نیست؛ نیازمند system callهای متعدد یا رابط جدید

در این پروژه، چون هدف مشاهده مستقیم ساختار داده پردازه‌ها در kernel بود، پیاده‌سازی در kernel انتخاب درستی است.

بخش سوم عملی: مرتب‌سازی در هسته و فضای کاربر

خواسته پروژه

باید system call زیر پیاده‌سازی شود:

```c
int sort_numbers(const char *src_file);
```

این فراخوانی باید نام فایل ورودی را بگیرد، محتوای آن را بخواند، اعداد موجود در فایل را مرتب کند، و نتیجه را در فایل جدیدی در همان مسیر ذخیره کند. ورودی شامل چند خط است و در هر خط یک عدد صحیح قرار دارد. اگر فایل ورودی وجود نداشته باشد، مقدار `-1` و در صورت موفقیت مقدار `0` برگردانده می‌شود. همچنین نباید از الگوریتم مرتب‌سازی بازگشتی استفاده شود.

پیاده‌سازی `sys_sort_numbers` در `sysfile.c`

در پروژه نهایی، `sys_sort_numbers` در `sysfile.c` پیاده‌سازی شده، چون با فایل‌سیستم کار دارد. مراحل آن:

1. گرفتن مسیر فایل با `argstr(0, &path)`.
2. پیدا کردن inode فایل ورودی با `namei(path)`.
3. ساخت یک `struct file` برای خواندن فایل.
4. خواندن محتوا در بافر `buf[4096]`.
5. عملیات parse کردن اعداد صحیح و ذخیره آن‌ها در آرایه `nums[1024]`.
6. مرتب‌سازی با insertion sort غیر بازگشتی.
7. ساخت نام خروجی با suffix زیر:

```text
.kernel.sorted
```

8. ساخت فایل خروجی با `create` و نوشتن هر عدد مرتب‌شده در یک خط.

نام خروجی برای فایل `nums` به صورت زیر خواهد بود:

```text
nums.kernel.sorted
```

الگوریتم مرتب‌سازی استفاده‌شده insertion sort است:

```c
for (i = 1; i < count; i++) {
    key = nums[i];
    j = i - 1;
    while (j >= 0 && nums[j] > key) {
        nums[j + 1] = nums[j];
        j--;
    }
    nums[j + 1] = key;
}
```

این الگوریتم بازگشتی نیست و بنابراین خطر مصرف بیش از حد kernel stack ندارد.

برنامه تست kernel sort: `sort_kernel.c`

برنامه زیر فایل ورودی را از آرگومان خط فرمان می‌گیرد، قبل و بعد از `sort_numbers` مقدار `uptime` را می‌خواند و زمان را بر حسب tick چاپ می‌کند:

```sh
sort_kernel nums
```

نمونه خروجی:

```text
kernel sort done in 3 ticks
```

پیاده‌سازی همان الگوریتم در user space: `sort_user.c`

برنامه `sort_user.c` همان کار را در فضای کاربر انجام می‌دهد:

1. فایل را با `open` باز می‌کند.
2. محتوا را با `read` می‌خواند.
3. اعداد را parse می‌کند.
4. همان insertion sort را اجرا می‌کند.
5. خروجی را در فایل جدید با suffix زیر می‌نویسد:

```text
.user.sorted
```

مثلاً برای فایل `nums` خروجی به صورت زیر خواهد بود:

```text
nums.user.sorted
```

روش اجرا:

```sh
sort_user nums
```

نمونه خروجی:

```text
user sort done in 1 ticks
```

نکته مهم: در کد نهایی، `sort_kernel.c` زمان کل system call شامل خواندن، parse، مرتب‌سازی و نوشتن را اندازه می‌گیرد، اما `sort_user.c` زمان بخش الگوریتمی `sort_ints` را جداگانه اندازه می‌گیرد. بنابراین هنگام مقایسه tickها باید این تفاوت را در نظر گرفت. برای مقایسه تعداد mode switchها، کل مسیر برنامه‌ها جداگانه تحلیل می‌شود.

قالب فایل ورودی و خروجی

فایل ورودی نمونه:

```text
5
1
9
-3
2
8
0
```

خروجی مرتب‌شده:

```text
-3
0
1
2
5
8
9
```

پاسخ پرسش‌های بخش sort

پرسش ۱: تعداد تغییر mode بین فضای کاربر و هسته در هر بخش چقدر است؟

هر system call حداقل دو تغییر mode دارد:

```text
user → kernel
kernel → user
```

حالت kernel sort

اگر فقط عملیات اصلی مرتب‌سازی را در نظر بگیریم، برنامه کاربر فقط یک system call اصلی صدا می‌زند:

```text
sort_numbers
```

پس تعداد تغییر mode برای عملیات اصلی:

```text
1 system call × 2 = 2 mode changes
```

اگر اندازه‌گیری زمان در `sort_kernel.c` را هم حساب کنیم، دو فراخوانی `uptime` هم وجود دارد:

```text
uptime + sort_numbers + uptime = 3 system calls
3 × 2 = 6 mode changes
```

اگر `printf` و `exit` را نیز حساب کنیم، تعداد system callها و mode changeها بیشتر می‌شود.

حالت user-space sort

در حالت user space، مرتب‌سازی داخل خود برنامه انجام می‌شود و خود الگوریتم insertion sort mode switch ندارد. اما خواندن و نوشتن فایل با system call انجام می‌شود. در برنامه نهایی system callهای اصلی عبارت‌اند از:

```
TEXT

open, read, close, uptime, uptime, open, write..., close
```

اگر تعداد writeها را `W` بگیریم، حداقل تعداد system callهای اصلی چنین است:

```
TEXT

open input + read + close input + uptime + uptime + open output + W writes + close output
= 7 + W system calls
```

پس تعداد mode changeها:

```
TEXT

2 × (7 + W)
```

در کد `sort_user.c` برای هر عدد یک بار `write` انجام می‌شود. اگر تعداد اعداد فایل `N` باشد، `W = N` و بنابراین:

```
TEXT

mode changes = 2 × (7 + N)
```

برای مثال، اگر فایل ۱۰۲۴ عدد داشته باشد:

```
TEXT

2 × (7 + 1024) = 2062 mode changes
```

این محاسبه نشان می‌دهد چرا انتقال کل عملیات به یک system call در kernel از نظر تعداد ورود و خروج از kernel سربار کمتری دارد؛ هرچند از نظر طراحی سیستم‌عامل، انجام sort در kernel معمولاً تصمیم خوبی نیست.

پرسش ۲: آیا لازم بود sort در فضای هسته پیاده‌سازی شود؟

خیر. مرتب‌سازی یک کار محاسباتی معمولی است و به دسترسی ممتاز kernel نیاز ندارد. در سیستم‌های واقعی، چنین کاری باید در user space انجام شود، چون:

- وجود bug در kernel می‌تواند کل سیستم را crash کند؛ اما bug در user space فقط همان برنامه را خراب می‌کند.
- پشته kernel stack محدود است و اجرای الگوریتم‌های پیچیده در kernel خطرناک‌تر است.
- بخش kernel باید کوچک، قابل اعتماد و مسئول مدیریت منابع باشد، نه انجام محاسبات عمومی برنامه‌ها.
- پیاده‌سازی قابلیت‌های application-level در kernel سطح حمله و پیچیدگی kernel را افزایش می‌دهد.

مزیت احتمالی kernel sort کاهش mode switchهاست، چون کل کار در یک system call انجام می‌شود. اما این مزیت برای کاری مثل sort معمولاً ارزش نقض اصل جداسازی user/kernel را ندارد. هدف این بخش آموزشی است: مشاهده تفاوت سربار system call و تجربه کار با فایل‌سیستم در kernel.

پرسش ۳: دستور `sort` در Linux در فضای کاربر پیاده‌سازی شده یا فضای هسته؟

دستور `sort` در Linux یک برنامه user-space است و در GNU Coreutils قرار دارد. این برنامه فایل‌ها یا stdin را می‌خواند، داده را در فضای کاربر مرتب می‌کند و خروجی مرتب‌شده را روی stdout یا فایل می‌نویسد. kernel فقط سرویس‌های عمومی مانند `open`، `read`، `write`، `mmap` و مدیریت فایل را فراهم می‌کند و الگوریتم sort در kernel نیست.

پرسش ۴: kernel مسئول چه کارهایی است؟

وظایف اصلی kernel عبارت‌اند از:

- مدیریت پردازه‌ها و زمان‌بندی CPU؛
- مدیریت حافظه، page tableها و فضای آدرس؛
- مدیریت فایل‌سیستم و انتزاع فایل؛
- مدیریت deviceها و driverها؛
- رسیدگی به exception، interrupt و trap؛
- ارائه system call به برنامه‌های user؛
- وقوع enforce در امنیت، جداسازی پردازه‌ها و کنترل دسترسی؛
- مدیریت منابع مشترک مثل RAM، CPU، دیسک و I/O.

پرسش ۵: چرا نباید از الگوریتم‌های مرتب‌سازی بازگشتی استفاده کنیم؟

الگوریتم‌های بازگشتی مثل quicksort یا mergesort در هر فراخوانی یک stack frame جدید ایجاد می‌کنند. در kernel، stack بسیار محدودتر از user space است. اگر ورودی بزرگ باشد یا recursion عمیق شود، kernel stack overflow رخ می‌دهد. پیامدهای احتمالی:

- خراب شدن حافظه kernel؛
- وقوع panic یا crash سیستم؛
- رفتار غیرقابل پیش‌بینی؛

- آسیب‌پذیری امنیتی.

به همین دلیل در این پروژه از insertion sort غیر بازگشتی استفاده شده است.

دستورهای ساخت و اجرای پروژه

ساخت پروژه

در ریشه پروژه:

```bash
make clean
make fs.img
```

در بررسی انجام‌شده، دستور `make fs.img` با موفقیت اجرا شد و `kernel`، برنامه‌های سطح کاربر و `fs.img` ساخته شدند. warning مربوط به `sh.c` درباره recursion از خود xv6 است و خطای build ایجاد نمی‌کند.

اجرای xv6

اگر QEMU نصب باشد:

```bash
make CPUS=1 qemu-nox
```

یا برای GDB:

```bash
make CPUS=1 qemu-nox-gdb
```

تست‌های پیشنهادی داخل xv6

```sh
getpidtest

dice 10

pinfo_test

# image های پشت سر هم یا با ویرایش echo یک فایل نمونه بسازید؛ مثلاً با
# سپس:
sort_kernel nums
cat nums.kernel.sorted

sort_user nums
cat nums.user.sorted
```

برای بررسی فایل ناموجود:

```sh
sort_kernel no_such_file
```

باید پیام خطا چاپ شود، چون `sort_numbers` مقدار `-1` برمی‌گرداند.

جمع‌بندی

در این پروژه، مسیر کامل اجرای فراخوانی سیستمی در xv6 بررسی شد: از wrapperهای `usys.S` و قرار دادن شماره system call در `eax`، تا trap، ساخت trapframe، اجرای `trap()` و dispatch در `syscall()`. سپس چهار system call جدید به xv6 اضافه شد.

در بخش اول، مولد اعداد شبه‌تصادفی با state مشترک kernel و محافظت spinlock پیاده‌سازی شد. در بخش دوم، اطلاعات پردازه مستقیماً از جدول پردازه‌ها استخراج و چاپ شد. در بخش سوم، مرتب‌سازی اعداد موجود در فایل هم در kernel و هم در user space پیاده‌سازی شد و تفاوت تعداد mode switchها تحلیل شد.

مهم‌ترین نکته طراحی این پروژه این است که هرچند انتقال کارهایی مثل sort به kernel می‌تواند تعداد system callها را کم کند، اما از نظر طراحی سیستم‌عامل واقعی، kernel باید تا حد امکان کوچک، امن و متمرکز بر مدیریت منابع باقی بماند. کارهای محاسباتی عمومی بهتر است در فضای کاربر انجام شوند.

منابع

1. صورت پروژه: `OS_Lab2.pdf`.
2. کد پروژه نهایی: `xv6-Lab2.zip`.
3. وب سایت Linux man-pages: `getrandom(2)` و `random(7)` برای توضیح RNG هسته Linux.
4. وب سایت Microsoft Learn: مستندات `BCryptGenRandom` برای RNG در Windows.
5. وب سایت GNU Coreutils Manual: مستندات دستور `sort`.
6. کد منبع xv6-public، فایل‌های `usys.S`، `syscall.c`، `trap.c`، `trapasm.S`، `proc.c` و `sysfile.c`.